

Agent 运行时 = 一段在「计算」与「通信」之间 反复切换的长循环

- Agent 不是"更大的模型", 而是带工具的长循环: 几十到上百次"LLM 推理 + 工具调用"往返
- 工具执行 (编译/测试/IO) 吃掉 **35%-61%** 请求时间, 编码任务高达 **60%** [来源: PASTE, arXiv 2603.18897] ✓
- 单→多 Agent 转折: 通信占比随规模上升, 渐成主导成本
- 多 Agent 比 Agent $\sim 2-5\times$ token/成本 (⚠ 数量级估算, 受任务/AI 数量/轮数影响) [来源: 承袭旧详细文档 P2-3]
- ARM 主场: NEON (鲲鹏 920 现役) + SVE/SVE2/SME (新一代鲲鹏 early access 现已可得) 是同一融合方向——武器已齐备, 团队可

三部分议程： **problem setup** → **diagnosis** → **prescription**

- **P1 基础 (8min)**: Agent 执行流程 = 计算 ↔ 通信的反复交织
- **P2 洞察 (10min)**: 单→多 Agent, 通信随规模上升、渐成主导 (现状 + 数据论证)
- **P3 融合 (9min)**: ARM 平台 (NEON 鲲鹏 920 现役 + SVE/SVE2/SME 新一代鲲鹏 early-access) 的通信计算融合 = 主场机会
- 全场一句话 spine: 通信占比随规模上升、逐渐压过计算成为主导成本

Agent 运行时 = loop 里被 LLM 光环掩盖的系统层

- 不碰：pre-training/fine-tuning、GPU 利用率、单次推理内核优化
 - FlashAttention 约 **2-4×** 加速 (HBM 读写 $O(N^2) \rightarrow O(N)$) [来源: arXiv 2205.14135] ✓
 - PagedAttention 让 vLLM 吞吐 **2-4×** [来源: arXiv 2309.06180] ✓
- 聚焦：感知→推理→规划→工具调用→记忆更新长循环里的"非 LLM"部分
 - CPU 调度（进程启动/prefork/池化）、内存访问（向量检索 gather/距离）
 - 工具通信（IPC：HTTP/UDS/共享内存）、序列化

Agent 不是"一次 API", 而是混合/突发/长尾的长循环

- 标准循环 (ReAct): 感知→推理→规划→工具调用→记忆更新→回到感知 [来源: ReAct, arXiv 2210.03629]
- 长循环权威数据: SWE-bench Verified 单实例上限 **250 步 / 3 美元** [来源: arXiv 2511.13646] ✓
- SWE-bench Pro 每任务 **50 次工具调用** ("相当标准") [来源: Scale AI, OpenReview 9R2iUHhVfr] ✓
- 每步不同计算形态: 感知(CPU 密集)、推理(远程→CPU 空闲)、规划(轻逻辑)、工具(CPU/IO 混合)、记忆(向量检索)

循环的每一步，是不同的计算形态、不同的资源需求

步骤	计算形态	资源需求	典型延迟
感知	文本处理/tokenization	CPU 密集、内存带宽	JSON 解析 GB/s 级
推理	LLM API 调用	网络 IO，本地 CPU 空闲	GPT-4o ~570ms、Claude ~671ms
规划	轻量推理/规则匹配	计算轻、逻辑复杂	μs-ms 级
工具调用	代码分析/profiler + IO	CPU/IO 混合、可能 fork	编译秒-分钟级、fork 1GB 6.5ms

主线第一次点题：loop = 计算 ↔ 通信的反复交织

- 把五步重画成 "本地计算 ↔ 跨边界通信" 的乒乓
 - 本地计算：感知、规划、记忆更新、工具前后处理
 - 跨边界通信：推理(LLM API)、工具调用(远程/IPC/文件网络 IO)
- 关键现象：API 等待期 (**500ms-5s+**,  数量级估算) 本地 CPU 利用率 **~0%** ( 串行阻塞场景)
- API 延迟：GPT-4o ~**570ms**、Claude ~**671ms**、GPT-4o Mini **200-400ms**、DeepSeek ~**23.64s** [来源: AILatency/SitePoint/Medium, 2024-2025]  数量级估算

Agent 是混合/突发/长尾/依赖复杂的新 workload

- **混合**: compute + comm 剧烈交替, 工具执行占 **35%-61%** (编码 60%) [来源: PASTE, arXiv 2603.18897] ✓
- **突发**: SWE-agent 成功 **12 步/\$1.21** vs 全部 (含失败) **21 步/\$2.52** [来源: arXiv 2405.15793] ✓
- **长尾**: Devin **72%** 成功任务耗时 **>10 分钟** [来源: Cognition SWE-bench Technical Report] ⚠ 数量级估算 (vendor self-report)
- **依赖复杂**: **55%** 文件编辑后立即跟随终端调用 (Edit-Verify 模式) [来源: PASTE, arXiv 2603.18897, §2.3.1] ✓

2026.04 全部重构假设 (单一计算/可预测时长/独立任务) **全部被打破** → 为

诊断问题：从 1 个 agent 到 N 个 agent，成本结构怎么变？

- 单 Agent 瓶颈在 loop 内部效率；多 Agent 瓶颈变成 多个 loop 之间的协调开销——即“通信”
- 新变量：“通信” = Agent 间消息传递、状态同步、角色调度、共识、负载均衡
- 本章诊断论断：通信占比随规模上升、逐渐压过计算成为主导成本
- 三类证据：学术基准（UIUC / Continuum）+ 框架拓扑（AutoGen/MetaGPT/CrewAI/LangGraph）+ 协议趋势（Google A2A）

A2A 生态分三层： 框架拓扑各异、A2A 协议标准化、学术基准量化成本

- 框架拓扑各自为政 [来源: AutoGen/MetaGPT/CrewAI/LangGraph 官方文档] 
 - AutoGen: 广播 / GroupChat pub-sub + GroupChatManager 选下一发言者
 - MetaGPT: 全局共享消息池 + **5 个顺序角色** (PM → Architect → PM → Engineer → QA) [arXiv:2308.00352, ICLR 2024] 
 - CrewAI: 层级任务委托; LangGraph: 有状态有向图 + checkpoint

这一页的任务：把三个数字的口径标清楚

- **~2-5× token/成本增量** (⚠ 经验法则/数量级估算, 指 token-or-cost 而非 **total-time**, 受任务/AI 数/轮数影响) [来源: 综合 UIUC arXiv:2505.18286]
 - 拓扑特定印证 (⚠ 绝非通用乘数): 中心化 supervisor-worker ≈ **+285%** ($\approx 3.85 \times$ 总量); 去中心化 +263%、混合 +515% [来源: arXiv:2512.08296 / Medium] ⚠ 数量级估算
- **~58% 调度气泡** (⚠ 调度气泡/任务间隙, 非工具调用时间——工具返回后等 KV-cache 重驻留的空闲) [来源: Continuum, arXiv:2511.02230] ✓
- **木桶效应**: 最慢 agent 决定端到端延迟——放大 fail-slow (SWE-agent 成功 **12 步/\$1.21** vs 全部 **21 步/\$2.52**) [来源: SWE-agent,

A2A 数据怎么流：四种模式 + 序列化税 + $O(N^2)$ 爆炸

- 四种通信模式：点对点(1-1) / 广播(1-N, 如 AutoGen) / 层级编排(树状, CrewAI) / 共享黑板(N-1 共享状态, MetaGPT)
- 序列化是隐藏的税：
 - JSON 比 Protobuf 慢约 **5-10×** (⚠ 数量级估算, 依赖语言/运行时/数据形态, 严谨基准多 3-6×) [来源: Zuplo/inomera, 2023-2024] ⚠
 - JSON 体积约 Protobuf 的 **6×** (FlatBuffers: 1475B vs 228B vs **FlatBuffers 344B**) [来源: FlatBuffers 官方 Benchmarks] ⚠

被锁与不均吃掉的延迟：锁竞争 + 共识墙 + 负载不均

- 锁竞争退化 **10-100×** (✅ 已确认；高竞争下临界区 = Amdahl 串行部分) [来源: ACCU Overload 2020 / EMSE 2017] ✅
 - 5-10 agents 频繁写约 **10-30%** 时间等锁；10+ agents 高并发写 **50%+** 等锁 (⚠️ 数量级估算，随工作负载)
- 分布式共识延迟墙 (⚠️ 数量级估算，随拓扑/存储介质):
 - etcd (Raft) LAN 单次提交约 **1-10ms** (fdatasync: HDD ~10ms、SSD <1ms) [来源: etcd 官方, 2017] ✅
 - Raft/Paxos 10 节点典型延迟 **10-50ms** 量级 [来源: 承袭旧详细文档]

诊断落地：通信占比随规模上升、渐成主导成本

单 agent → 小 swarm → 大 swarm，通信从配角变成主角

规模	通信表现	数据
单 agent	占比小，瓶颈在 loop 内部	工具执行 35%-61% 
小 swarm	通信上升，序列化税/ $O(N^2)$ 显现	成本增量 ~2-5×  经验法则
大 swarm	通信占比主导	气泡 ~58% / 锁 10-100× / 共识 10-50ms

通信变贵了，谁来付？（handoff——只问不答）

“如果通信是上升中的主导成本，而我们 ARM 平台（NEON 鲲鹏 920 现役 + SVE/SME 新一代鲲鹏 early-access 现已可得）能把通信和计算放进同一组 SIMD 单元……”

- 传统架构：comm 的"数据搬运/序列化"与 compute 的"向量化/矩阵化" = 两套单元、两次拷贝，边界开销无法消除
- 融合架构：序列化/pack/gather/转置本身就是 SIMD/矩阵操作——同一组单元、同一份布局、无边界拷贝
- ARM 物理基础：NEON（鲲鹏 920 现役）+ SVE/SVE2/SME（新一代鲲鹏 early-access,  暂无公开来源）[来源：

通信计算融合 = 主场（handoff 落地）

承接 P2-7 的张力——"通信变贵了，谁来付？"——答案是融合。

- 不再把通信与计算当成两个独立问题、两套单元、两次拷贝
- 合并成同一类操作、同一组单元、同一份布局
- 现役 **NEON** 已能做基础融合（向量化序列化 / pack / 基础 SIMD）——鲲鹏 920 立即可做
- **SVE predicate+gather/scatter** 与 **SME 矩阵引擎** 在新一代鲲鹏 early-access 现已可得（⚠ 暂无公开来源）；NEON 现役与 SVE/SME 是同一融合方向的现在与未来
- 前后是同一融合主线的现在与未来，不是另起炉灶

P3-2 三类武器： NEON / SVE / SME — 为什么对 Agent 特别

Agent 负载是变长 / 不规则 / 小批量——三类单元的结构性匹配。

单元	状态	对 Agent 的匹配
NEON	● 现役（鲲鹏 920）	固定 128-bit SIMD；无 per-lane 谓词、无原生 gather/scatter
SVE / SVE2	● 新一代鲲鹏 early-access 已可得（⚠ 暂无公开来源）	VLA 128-2048 bit + predicate + 原生 gather/scatter
	●	

P3-3 通信计算融合 = 三件事 (thesis 展开)

融合不是一句口号，是三件具体的事——今天 NEON（鲲鹏 920 现役）就能立论，SVE/SME（新一代鲲鹏 early-access）现已可得。

① 重叠 — 通信期间做计算（延迟隐藏）

- 经典 HPC：MPI 非阻塞 **Isend/Irecv**，计算与传输并发 [来源：Bernholdt 等, OSTI 944757, 2008] 
- API 等待 **500ms-5s+** 期间本地 CPU 不必空闲，可 SIMD 做向量检索/rerank/模板渲染
-  重叠不免费：GPU 实测计算减速平均 **~18.9%**（最高 **40%**）；纯顺序更慢平均 **10.2%**（ GPU 特定，CPU 仅定性类比） [来源：Georgia Tech, ISPASS 2025, arXiv:2507.03114] 

P3-3 ③ **kernel + 原语协同**：同一组单元、同一份布局（→ 融合点）

③ **kernel + 原语协同** – comm 原语与算子共享同一组 **SIMD** 单元 + 同一份布局，消除边界拷贝；iceoryx2 零拷贝 IPC = "不搬运载荷"的极端端点（⚠ sub-μs、与 payload 无关；勿暗示快于 **raw shmem**）[来源：iceoryx2 Discussion #435] ⚠

P3-4 算子侧 · NEON 现役：专用小-GEMM kernel

Agent 算子 (embedding / rerank / 小批量 GEMM / attention) 是**"瘦高"矩阵**，batch **1-8** vs 训练 **32-1024**——结构性不同于大方阵批量 GEMM [来源: 承袭 P3-4 校验] ⚠️。

NEON 现役 — 基础向量化 + 专用小-GEMM kernel

- **LibXSMM** (SC15): 小 GEMM 加速 **>10×** (可达/上限)，典型 MKL 对比 **2-4×** ("取决于 CPU 与应用"，无鲲鹏 920 专用数字) [来源: SC15] ✓

2026-06-06 二部讨论稿
• **LibShalom** (SC'21): **target ARMv8 多核含鲲鹏 920**; 小 M 时

P3-4 算子侧 · SME early-access: outer-product + ZA tile (⚠ 非万能药)

SME (新一代鲲鹏 early-access, ⚠ 暂无公开来源) — outer-product + ZA tile (⚠ 不是万能药, 需混合策略)

- KirbyMM (DATE 2026): SME MOPA 峰值 **0.512 TFLOPS FP32/核** (稠密 GEMM 实测仅约 0.33) vs SVE MLA 0.128 = **4×**; 但稠密 GEMM 因此内存带宽受限
- KirbyMM 边缘 case: 专用 SME+SVE 混合平均比 KBLAS 快 **3.59×** (LX2 ARMv9/SME, 5 个瘦 case FP32, 非当前 **920**) [来源: DATE 2026] ✓

P3-5 通信侧 · 序列化加速：NEON 现役 simdjson 式 GB/s 解析

A2A / 工具调用流量绝大多数是 **JSON** 形状——序列化/验证数据通路是现实瓶颈，远早于 LLM 推理成本。

序列化加速 — NEON 现役 simdjson 式 GB/s 解析

- simdjson: 首个单商用核 **GB/s** 级完全校验 JSON 解析器；指令数仅 RapidJSON 的 **1/4 或更少**；约 **4×** 快于 RapidJSON、约 **25×** 快于 nlohmann/json [来源: Langdale & Lemire, VLDB Journal 28(6) 2019, arXiv:1902.08318] 

P3-5 通信侧 · 消息搬运：零拷贝 IPC + AoS→SoA 转置

零拷贝 IPC — iceoryx2 (⚠ 数量级, 定性)

- 常数级 **sub-μs**, 与 payload 大小无关 [来源: iceoryx2 Discussion #435] ⚠
- ⚠ 勿暗示快于 **raw shmem** (同来源: 单字节反而慢约 600ns)

消息搬运 — NEON AoS→SoA 转置 (鲲鹏 920 现役) / SVE gather-scatter (新一代鲲鹏 early-access)

- **NEON 无 gather/scatter** (仅 LD1-LD4 连续/交错); 非连续 AoS 字



P3-6 融合点：同一组单元、同一份布局

(prescription climax)

传统架构 vs 融合架构——这是整场主线第三次点题、climax。

	传统架构	融合架构
单元	两套：comm 栈（HTTP/JSON/拷贝） + compute 栈（向量化/矩阵）	同一组（NEON 鲲鹏 920 现役）
布局	两份独立 → 边界 AoS ↔ SoA 转置拷贝	同一份（SoA + 连续 ld1/st1）
边界	拷贝/序列化/布局转换结构性存在、无法清除	无边界拷贝

P3-7 怎么开始：三层机会 + 路线图

时间线明确，短期 NEON（鲲鹏 920 现役部署）立即可做 + SVE/SME（新一代鲲鹏 early-access）实测启动，无需等任何未来平台。

阶段	时间	平台 / 口径	动作
短期	现在-6月	● NEON 现役（鲲鹏 920 立即） + ● SVE/SME（新一代鲲鹏 early-access 实测启动）	NEON: simdjson 式解析 + iceoryx2 零拷贝 IPC 原型 + vzip/vuzp/vtrn pack + 小-GEMM kernel; SVE/SME 实测: SVE gather/scatter 序列化 + SME outer-product micro-bench + 小-

总结 + 讨论问题（收束）

整场一条主线：**P1** loop = compute ↔ comm 交织 → **P2** 通信随规模上升、渐成主导 → **P3** 在 ARM 平台把 comm 与 compute 融合 = 主场。

核心收束 — 通信计算融合就是主场

- **NEON**（鲲鹏 920 现役）今天就能做：simdjson 式 GB/s、vzip/vuzp/vtrn pack、小-GEMM kernel（LibXSMM/LibShaLom 风格）、iceoryx2 零拷贝 IPC——鲲鹏 920 立即可做
- **SVE/SME** 是同一融合方向的延续（新一代鲲鹏 early-access 已可得，⚠️ 暂无公开来源，他平台数字需在本团队硅 re-measure）
- **团队能力高度匹配**：GEMM/Attention + MPI/UCX/SDMA + 性能方